Operating system II

Part IV

D E A D L O C K S

K.Belarbi

University of Science and Technology Mohamed Boudiaf Oran
Mathematics and Computer Science Faculty
Computer Science Department

April 24th, 2025

Table of contents

1. Introduction
   - Definition
2. Deadlock Characterization
3. The ostrich algorithm
4. Deadlocks detection and recovery
5. Deadlock Avoidance
6. Deadlock Prevention

Introduction

- Les ordinateurs ont de nombreux ressources qui ne peuvent servir qu'à un seul process à la fois.
- un process ne peut utiliser une ressource qu'en suivant la séquence suivante :
  - **La requête** : le process fait une demande pour utiliser la ressource. Si cette demande ne peut pas être satisfaite immédiatement, parce que la ressource n'est pas disponible, le process demandeur se met en état attente jusqu'à ce que la ressource devienne libre.
  - **Utilisation** : Le process peut exploiter le ressource.
  - **Libération** : Le process libère la ressource qui devient disponible pour les autres process éventuellement en attente.

Introduction

Definition

Qu'es qu'un Deadlock ?

Introduction

Definition

Deadlock

*Un ensemble de process est en **Deadlock** si chaque process attend un événement que seul un autre process de l'ensemble peut provoquer.*

- Tout les process attendent –> aucun événement n'est produit –> Tout les process attendent indéfiniment ;
- Aucun process n'est réveillé ;
- La libération d'une ressource détenue par un autre est un événement ;
- Le nombre des process et le nombre et le type des ressources (matérielles où logicielles) détenues et sollicitées est sans importances ;
- On dit **deadlock between resources** (inter-blocage de ressources)

1 Introduction

2 Deadlock Characterization
- Conditions for Resource Deadlocks
- Deadlock modeling

3 The ostrich algorithm

4 Deadlocks detection and recovery

5 Deadlock Avoidance

6 Deadlock Prevention

Deadlock Characterization

Conditions for ressource Deadlock

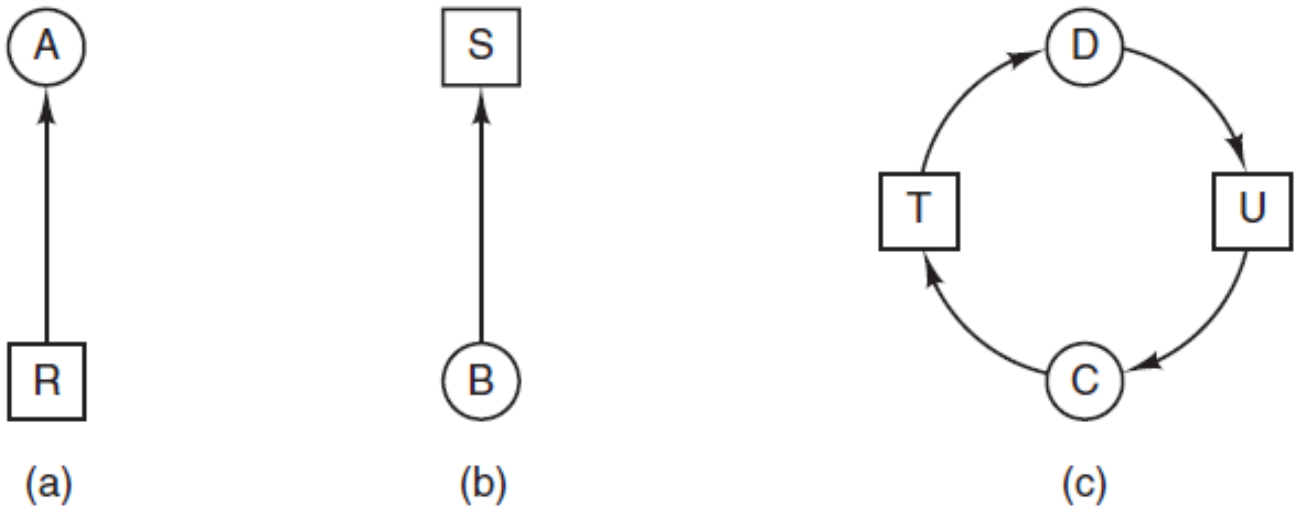
Pour provoquer un Deadlock, quatre conditions sont à satisfaire : (Coffman, Elphick et Shoshani)

- **Mutual exclusion** : une ressource est soit allouée à un seul process, soit disponible ;
- **hold and wait** (Détention et attente) : les process qui détiennent des ressources peuvent en demander d'autres ;
- **no "preemption"** : les ressources allouées à un process sont libérées uniquement par le process (ressources non préemptives) ;
- **Circular wait** : un ensemble de process attendant chacun une ressource allouée à un autre.
- **Exemple** : les cinq philosophes ont chacun la fourchette de gauche, et attendent la fourchette de droite.

- Le graphe d'allocation de ressources est un graphe orienté (Holt, 1972) ;
- Il montre pour chaque process les ressources qu'il possède et celles dont il a besoin.
- C'est un graphe biparti avec deux types de nœuds et un ensembles d'arcs :

- *les process* : illustrés par des cercles ;
- *les ressources* : représentées par des carrés ;
- *un arc orienté d'une ressource vers un process* signifie que la ressource est allouée au process ;
- *Un arc orienté d'un process vers une ressource* signifie que le processus est bloqué en attente de la ressource.

Figure – **Resource allocation graphs**. (a) *Holding a resource*. (b) *Requesting a resource*. (c) *Deadlock*.



Exemple1 :
Soient trois process A, B et C qui utilisent trois ressources R, S et T

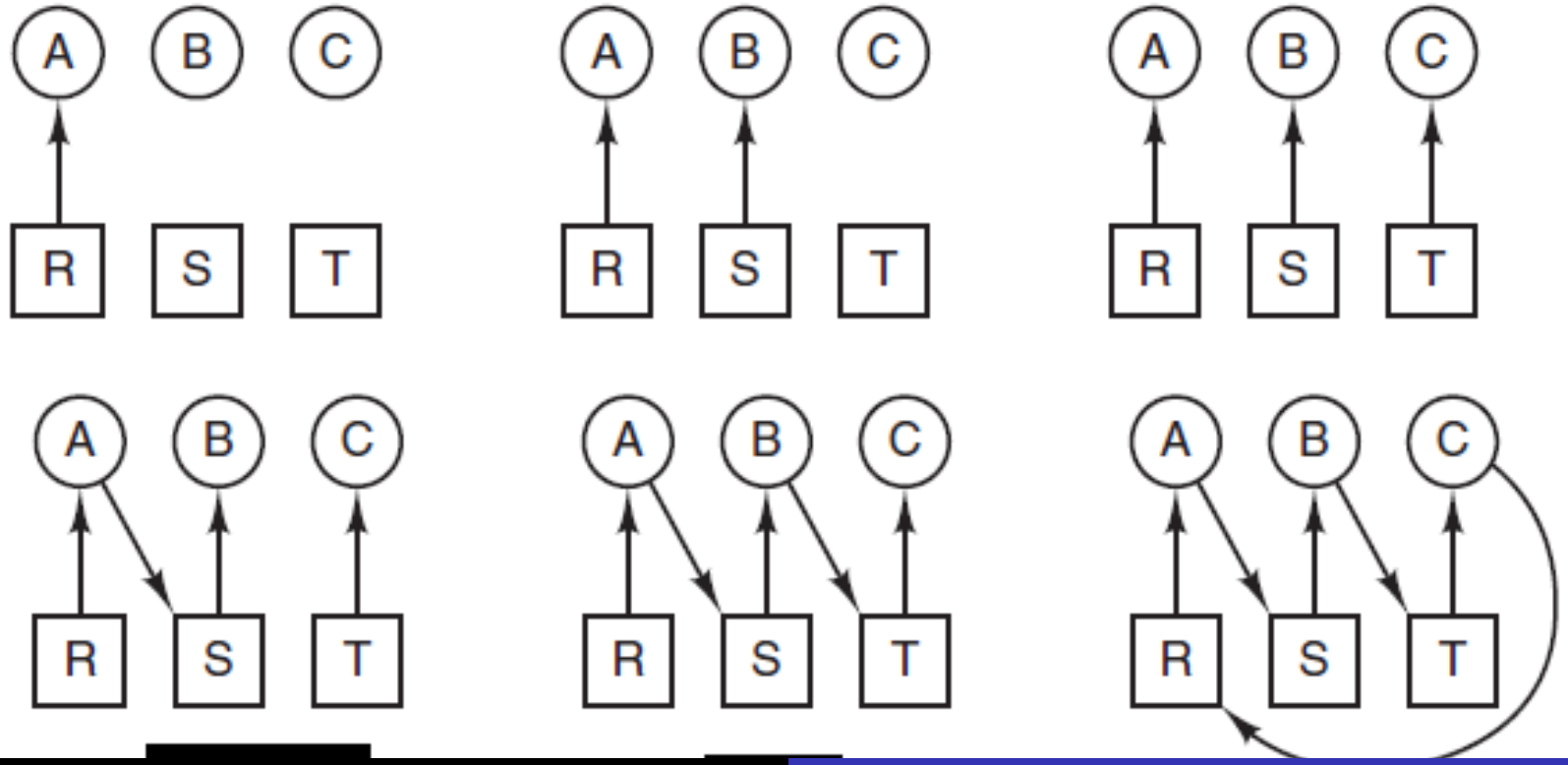
A	B	C
Request R	Request S	Request T
Request S	Request T	Request R
Release R	Release S	Release T
Release S	Release T	Release R

- Si les process sont exécutés séquentiellement **A** suivi de **B** suivi **C**, il n'y aurait pas Deadlock.
- On atteindrait une situation deadlock, si les instructions sont exécutées dans cet ordre :

Deadlock Characterization

Deadlock modeling

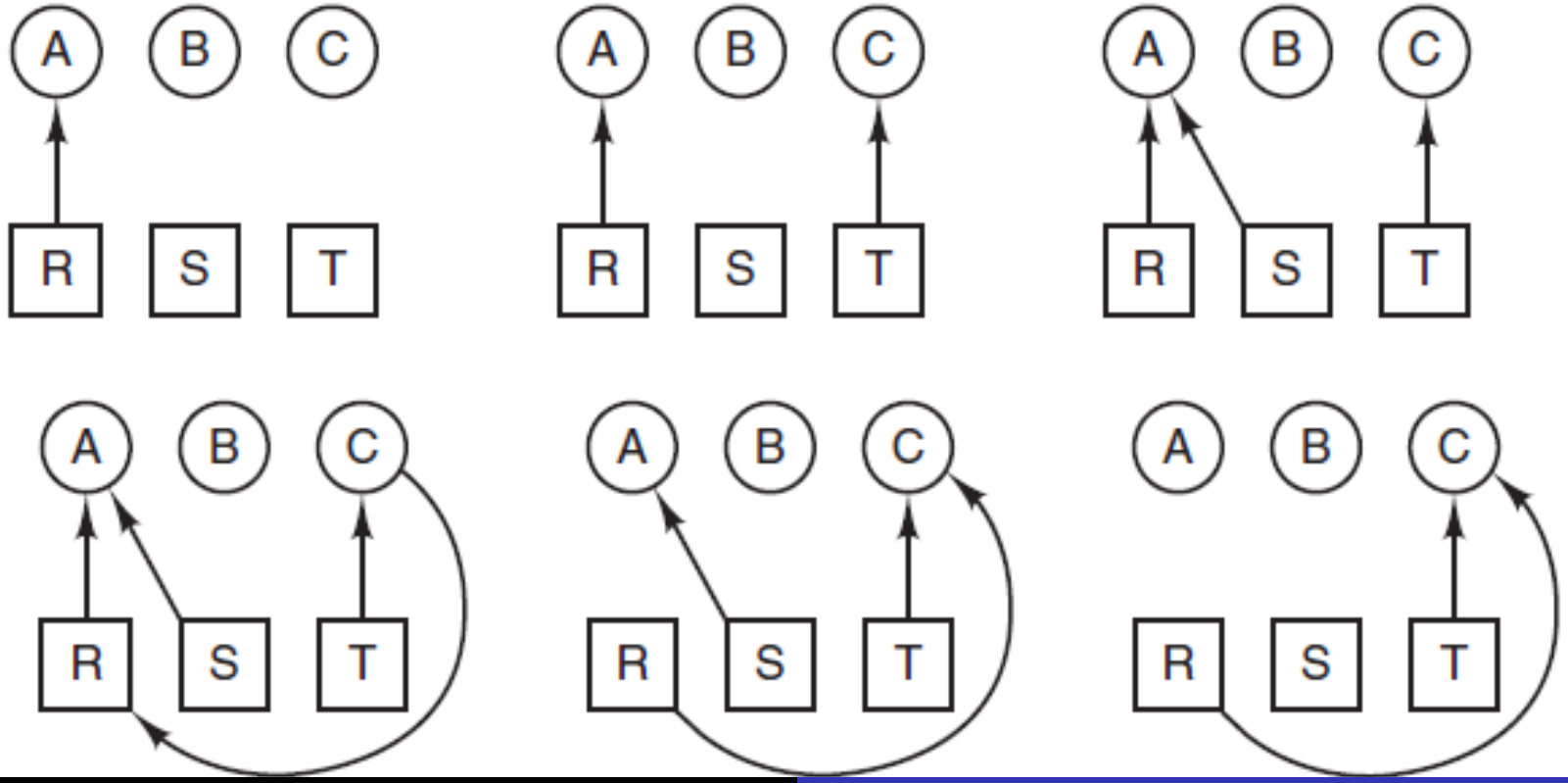
1. A requests R
2. B requests S
3. C requests T
4. A requests S
5. B requests T
6. C requests R



Deadlock Characterization

Deadlock modeling

1. A requests R
2. C requests T
3. A requests S
4. C requests R
5. A releases R
6. A releases S



Deadlock Characterization

Solutions au problème deadlock

Pour gérer les deadlocks, on peut procéder de quatre façons :

1. Ignorer les problèmes
2. les détecter et y remédier
3. Les éviter de manière dynamique en allouant les ressources avec précaution. Si l'allocation d'une ressource peut conduire à un deadlock, elle est retardée jusqu'à ce qu'il n'y ait plus de risque.
4. Les prévenir en empêchant l'apparition de l'une des quatre conditions nécessaires à leur existence.

1 Introduction

2 Deadlock Characterization

3 The ostrich algorithm

4 Deadlocks detection and recovery

5 Deadlock Avoidance

6 Deadlock Prevention

The ostrich algorithm

La solution la plus simple.

Prétendre qu'il n'y a aucun problème !

En général, ce problème est ignoré par la plus part des systèmes d'exploitation car le prix à payer pour les éviter ou les traiter est trop élevé pour des situations qui se produisent rarement.
Raisonnable comme solution si les deadlocks sont rares. Elle est facile à comprendre et a mettre en œuvre.

1 Introduction

2 Deadlock Characterization

3 The ostrich algorithm

4 Deadlocks detection and recovery
- Deadlocks detection algorithm
- Deadlock Recovery

5 Deadlock Avoidance

6 Deadlock Prevention

Deadlocks detection and recovery

Deadlocks detection

(a) Construction du graphe des ressources : Si graphe contient un ou plusieurs cycles alors Deadlock
–> tout process faisant partie d'un cycle est bloqué.
S'il n'y a aucun cycle, le système n'est pas bloqué.

(b) Les systèmes actuelles nécessitent des algorithmes de détection deadlock.

Deadlocks detection and recovery

Deadlocks detection algorithm

Soient :
n le nombre de processus $P[1], P[2], ..., P[n]$
m le nombre de types de ressources $E[1], E[2], ..., E[m]$
L'algorithme de détection des Deadlock utilise les matrices et vecteurs suivants :

- Matrice **C** des allocations courantes d'ordre *(n x m)*. L'élément *C(i,j)* désigne le nombre de ressources de type $E[j]$ détenues par le processus $P[i]$.
- Matrice **R** des demandes de ressources d'ordre *(n x m)*. L'élément $R[i, j]$ est le nombre de ressources de type $E[j]$ qui manquent au processus $P[i]$ pour pouvoir poursuivre son exécution.
- Vecteur **A** d'ordre m . L'élément $A[j]$ est le nombre de ressources de type $E[j]$ disponibles(non attribuées).
- Vecteur **E** d'ordre m . L'élément $E[j]$ est le nombre total de ressources de type j existantes dans le système.

Deadlocks detection and recovery

Deadlocks detection algorithm

1. Rechercher un process $P[i]$ non marqué dont la rangée $R[i]$ est inférieure à **A**
2. Si ce process existe, ajouter la rangée $C[i]$ à **A**, marquer le process et revenir à l'étape 1
3. Si ce process n'existe pas, les process non marqués sont en deadlock. L'algorithme se termine.

Deadlocks detection and recovery

Recovery

1. **reprendre au moyen de la préemption** : Possibilité de retirer provisoirement une ressource à son propriétaire pour l'attribuer à une autre process. (intervention manuelle, Os par lots). Cette possibilité dépend fortement de la nature de la ressource ainsi que le choix des process à suspendre.
2. **Reprendre au moyen du rollback** : Restauration d'un état antérieur du process – image de la mémoire et l'état de la ressource – points de reprises
3. **Reprise en moyen de la suspension des process***(plus simple)* : supprimer un ou plusieurs process. Il est préférable de choisir un process qui peut être redémarré depuis le début sans conséquence.

La reprise n'est pas évidente

1 Introduction

2 Deadlock Characterization

3 The ostrich algorithm

4 Deadlocks detection and recovery

5 Deadlock Avoidance
- Safe and Unsafe State

6 Deadlock Prevention

Deadlock Avoidance

- Lorsqu'un process demande une ressource, le système doit déterminer si l'attribution de la ressource est sûre (mène vers un état sûr).
  - Si c'est le cas, il lui attribue la ressource.
  - Sinon, la ressource n'est pas accordée.
- Un état est sûr si tous les process peuvent terminer leurs exécutions (il existe un ordre d'allocation de ressources qui permet à tous les process de se terminer).
- Il faut connaître à l'avance les besoins en ressources de chaque process (ce qui est en général impossible).

Deadlock Avoidance

Safe State

Comment déterminer si un état est sûr ou non sûr ?

Djikstra a proposé en 1965 un algorithme d'ordonnancement, appelé **Banker's Algorithm** (Dijistra, 1965 [1], Habermann 1969 [2]) qui permet de répondre à cette question. Cet algorithme consiste à examiner chaque nouvelle requête pour voir si elle conduit à un état sûr. Si c'est le cas, la ressource est allouée, sinon la requête est mise en attente.

1. Dijkstra, E.W., 1965. Solution of a Problem in Concurrent Programming Control. Communications of the ACM, 8(9) :569, 1965.
2. Habermann, A.N. Prevention of System Deadlocks. CACM, Vol. 12, No. 7, July 1969,pp. 373-385.

Deadlock Avoidance

Safe State

Cet algorithme, utilise des matrices A,E,C et R. Un état est caractérisé par ces quatre tableaux.

1. Trouver un process P_i non marqué dont la rangée i de R est inférieure à A.
2. Si un tel processus n'existe pas alors l'état est non sûr (il y a Deadlock). L'algorithme se termine.
3. Sinon, ajouter la rangée i de C à A, et marquer le processus.
4. Si tous les processus sont marqués alors l'état est sûr et l'algorithme se termine, sinon aller à l'étape 1.

Banker's algoritm permet bien d'éviter les Deadlock mais il est peu utilisé en pratique car on ne connaît pas toujours à l'avance les besoins en ressources des processus.

1 Introduction

2 Deadlock Characterization

3 The ostrich algorithm

4 Deadlocks detection and recovery

5 Deadlock Avoidance

6 Deadlock Prevention

Deadlock Prevention

Briser une des quatre conditions :

- **Mutual Exclusion**. Pour éviter l'exclusion mutuelle, il est parfois possible de sérialiser les requêtes portant sur une ressource. Par exemple, pour les imprimantes, les processus « spoolent » leurs travaux dans un répertoire spécialisé et un démon d'impression les traitera, en série, l'un après l'autre.
- **Hold and Wait**. Pour ce qui concerne la deuxième condition, elle pourrait être évitée si the process demandait ces ressources à l'avance. Ceci est en fait très difficile à réaliser dans la pratique car l'allocation est, en général, dynamique. Empêcher cette condition serait donc particulièrement coûteux.

Deadlock Prevention

- **No preemption**. La troisième condition n'est pas raisonnablement traitable pour la plupart des ressources sans dégrader profondément le fonctionnement du système. On peut cependant l'envisager pour certaines ressources dont le contexte peut être sauvegardé et restauré.
- **Circular wait**. Résoudre le problème de l'attente circulaire en numérotant les ressources et en n'autorisant leur demande, par un processus, que lorsqu'elles correspondent à des numéros croissants ou en accordant aux processus une seule ressource à la fois (s'il a besoin d'une autre ressource, il doit libérer la première).
  En général, le nombre potentiel de ressources est si important qu'il est très difficile de trouver la bonne fonction F pour les numéroter